

THE AUSTRALIAN NATIONAL UNIVERSITY  
*First Semester Examination – June 2016*

**COMP3320/6464/HONS**

**High Performance Scientific Computing**

*Study Period: 15 minutes*

*Time Allowed: 3 hours*

*Permitted Materials: Non-programmable calculator,  
paper-based dictionary, one A4 page with notes on both sides*

*Exam questions total 100 marks.*

*All students answer all questions*

*Clarity and conciseness in answers will be highly valued  
Marks may be lost for supplying irrelevant information*

## Question 1 [20 marks]

### Short Questions

- (a) (i) Explain why the divergent infinite series:

$$\sum_{n=1}^{\infty} \frac{1}{n}$$

can have a finite sum in floating-point arithmetic.

- (ii) At what point would the partial sum cease to change?

[4 marks]

- (b) Give three examples of events you might seek to measure using hardware performance counters, stating why each event is useful to measure.

[3 marks]

- (c) In the following, `x` and `y` are double precision floating point arrays, while `a`, `b`, and `c` are double precision floating point scalar variables.

(i) `for (i=0; i<n; i++) a = a + x[i]*x[i];`

(ii) `for (i=0; i<n; i++) x[i] = (a*y[i]+b*y[i])*c;`

(iii) `for (i=0; i<n; i++) x[i+1] = x[i] + y[i];`

Order the above loops according to the time you think they would take on a contemporary x86 processor such as the `raijin` system used in this course. Provide a detailed explanation for your answer.

[6 marks]

- (d) Out-of-order execution is considered to be one of the key features of post-RISC architectures. What is out-of-order execution and how does it relate to register renaming?

[2 marks]

- (e) Give a few lines of code to indicate what is meant by

(i) A control dependency

(ii) A data dependency

(iii) An output dependency

[3 marks]

- (f) In shared memory parallel computing what is a false cache line conflict? How does it affect performance, and how would you seek to remove it.

[2 marks]

## Question 2 [20 marks]

### *Shared Memory Programming with OpenMP*

- (a) Consider three distinct  $N \times N$  matrices, A, B and C. The following code performs a matrix multiplication using a single thread of execution.

```
for ( I = 0 ; I < N ; I++)
    for ( J = 0 ; J < N ; J++) C[I][J] = 0;

for ( I = 0 ; I < N ; I++)
    for ( J = 0 ; J < N ; J++)
        for ( K = 0 ; K < N ; K++) C[I][J] += A[I][K]*B[K][J] ;
```

Two attempts to develop an OpenMP parallel implementation are made:

#### Parallel Version #1

```
#pragma omp parallel
{
    #pragma omp for nowait
    for ( I = 0 ; I < N ; I++)
        for ( J = 0 ; J < N ; J++) C[I][J] = 0;

    #pragma omp for
    for ( I = 0 ; I < N ; I++)
        for ( J = 0 ; J < N ; J++)
            for ( K = 0 ; K < N ; K++) C[I][J] += A[I][K]*B[K][J] ;
}
```

#### Parallel Version #2

```
#pragma omp parallel
{
    #pragma omp for
    for ( I = 0 ; I < N ; I++)
        for ( J = 0 ; J < N ; J++) C[I][J] = 0;

    #pragma omp for
    for ( K = 0 ; K < N ; K++)
        for ( J = 0 ; J < N ; J++)
            for ( I = 0 ; I < N ; I++) C[I][J] += A[I][K]*B[K][J] ;
}
```

You test both implementations using multiple threads on a variety of different computer systems. You find that under some circumstances both versions give **incorrect** results.

- (i) Explain why each parallel implementation can sometimes give **correct** results, but can also sometimes give **incorrect** results. Describe the hardware/runtime circumstances where each scenario is likely to hold.

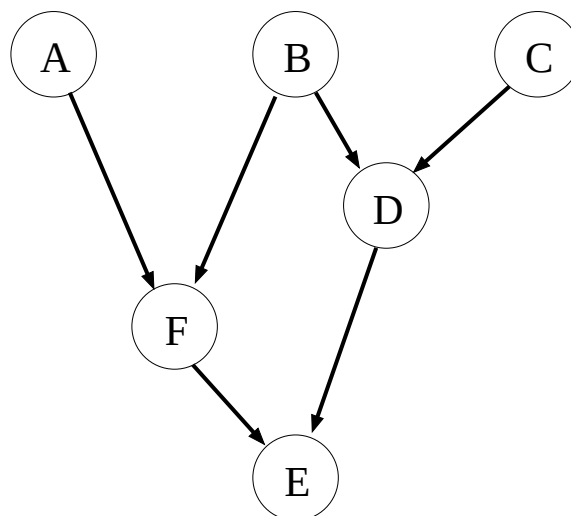
- (ii) In each case what modifications to the OpenMP directives would you make to ensure that both parallel implementations are always correct. (You should make **no** modifications to the basic loop structure, but are free to move, rewrite or add new OpenMP directives.)
- (iii) Discuss the performance of your two correct OpenMP versions. You should consider issues like the number of threads used, the problem size etc. Define, as required details of the parallel computer system you are running on.

[16 marks]

(b) Suppose that you had six C functions

```
void functionA(void)
void functionB(void)
void functionC(void)
void functionD(void)
void functionE(void)
void functionF(void)
```

that together solve some problem (since there are no parameters or return values, you should assume that the functions modify globally visible variables). These functions depend on each other according to the following dependency graph:



where, for example, **functionA** must complete execution before **functionF** can start. Sketch a C program that uses OpenMP to execute the above six functions in a way that is maximally parallel, but that adheres to the above dependency graph. (Hint, consider use of OpenMP sections.)

[4 marks]

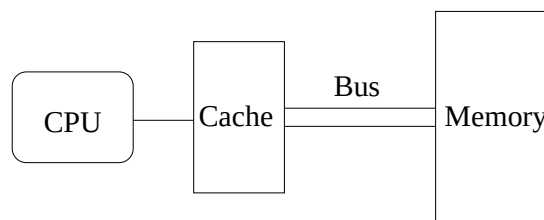
### Question 3 [20 marks]

#### *High Performance Computer Architecture*

- (a) (i) In the context of modern microprocessors what is pipelining?  
(ii) Under what circumstances, if any, is it better to have a 5 stage pipeline with each stage taking 2ns, compared to a 15 stage pipeline with each stage running at 1ns? Under what circumstances is the opposite true?  
(iii) What is software pipelining? Include in your answer an example.

[10 marks]

- (b) Consider the following memory system:



- The cache line size is 4 words
- The cache uses a write allocate policy, i.e. on a write miss the relevant cache line worth of data must first be read from memory into the cache before the write can proceed.
- The cache is write through, i.e. the store of a word propagates that change through to main memory. (Only the word is propagated, not the entire cache line).
- The bus can transfer data to or from memory at a rate of  $10^6$  words/sec. (Only in one direction at any given time.)

The following behaviour is observed when running a certain application.

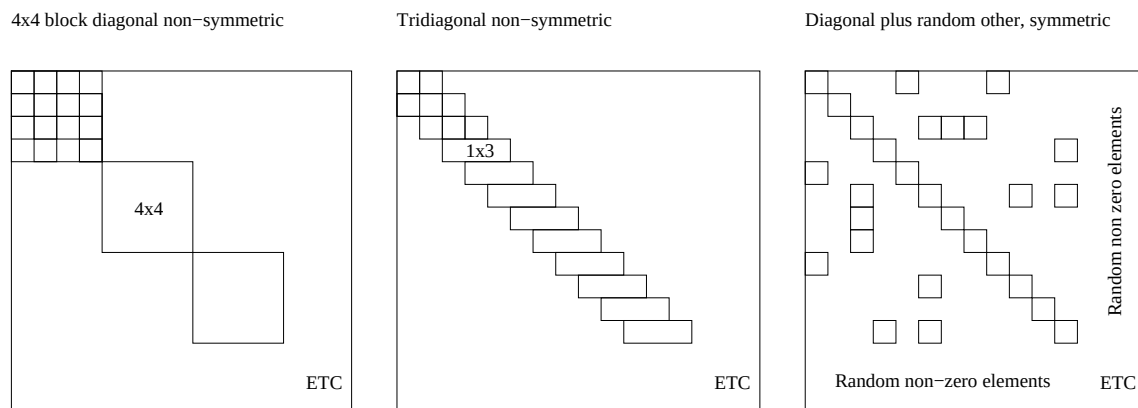
- The CPU sends memory requests (loads or stores of one word) to the cache at a rate of  $10^6$  requests/sec, with 25% of the requests being writes.
  - 90% of all memory requests issued by the CPU are resolved by data already in the cache (i.e. the cache hit ratio is 90%).
- (i) Given the above compute the bus traffic generated (in words/sec) by the following events:
- Read hits.
  - Read misses.
  - Write hits.
  - Write misses.
- (ii) What percentage of the total available bus bandwidth ( $10^6$  words/sec) is used?
- (iii) What additional information do you require in order to estimate the percentage of the total available bus bandwidth used if the cache were write back?

[10 marks]

## Question 4 [20 marks]

### *Application of High Performance Computing*

Consider three sparse square matrices. The first has a block diagonal structure, where each block is of size 4x4 and the matrix is not symmetric. The second is tridiagonal and also not symmetric, i.e the diagonal and the elements immediately above and below the diagonal are non-zero and different. The third is symmetric with a non-zero diagonal element and roughly 5% of the other elements also non-zero with arbitrary number of non-zero elements on each row. This is illustrated below:



You have been asked to write high performing code to compute a matrix vector product for each of the above matrices. Using  $A$  to denote the sparse matrix,  $N$  the dimension of  $A$ ,  $x$  the input vector, and  $b$  the product (i.e.  $b = Ax$ ) answer the following questions:

- For each matrix type, show how you would store elements of the matrix in memory. Explain why you use each storage pattern and give (in terms of  $N$ ) the total memory associated with storing matrix  $A$ .  

**[4 marks]**
- For each matrix type sketch 3 different pieces of C code to show how you would evaluate the matrix vector product. (Assume you are running on a single core of an x86 processor and do not include explicit use of SSE intrinsics functions.)  

**[6 marks]**
- In terms of  $N$  what is the computational cost associated with each of your matrix vector product implementations? Comment on how well you expect each to perform (in terms of fraction of peak). You should consider issues such as the ability of the compiler to use SSE instructions, cache usage, etc.  

**[4 marks]**
- For each of your three implementations, show how you would parallelise it using OpenMP. Comment on the likely performance of your code on both Uniform Memory Access (UMA) and Non-Uniform Memory Access (NUMA) parallel computers.  

**[6 marks]**

## Question 5 [20 marks]

### *Performance Modelling*

- (a) The scalability of a parallel code can be limited by i) algorithmic limitations, ii) bottlenecks, iii) startup overheads, and iv) communication. For each case give an example to illustrate that type of scalability limitation (i.e. four different examples).

[4 marks]

- (b) The speedup ( $S$ ) of an OpenMP parallel code running on  $N$  identical cores can be modelled as

$$S(N) = \frac{1}{f + (1 - f)/N + \kappa N + \lambda}$$

where  $f$  is the fraction of sequential code and  $\kappa$  and  $\lambda$  are constants.

- (i) When  $\kappa$  and  $\lambda$  are zero this equation is identical to Amdahl's law. What is the justification for the inclusion of the  $\kappa$  and  $\lambda$  terms?
- (ii) Draw a graph showing how  $S(N)$  varies as a function of  $N$ . Make clear what assumptions you make regarding the values of the other quantities in the equation. Highlight the key points of interest in your graph.
- (iii) If  $f = 10\%$ , estimate how many threads would be required to achieve  $S_{1/2}$ , where  $S_{1/2}$  is half the maximum possible speedup? Show how you derive your answer.

[10 marks]

- (c) In 1991 David Bailey wrote a famous article entitled "Twelve Ways to Fool the Masses When Giving Performance Results on Parallel Computers". This was updated in 2010 by Georg Hager to "Thirteen modern ways to fool the masses..". Three of the ways are

1. Report scalability, not absolute performance
2. If you must report actual performance, quietly employ weak scaling to show off.
3. Always quote GFlops, MIPs, Watts per Flop or any other "interesting" metric instead of inverse time to solution

For each of the above explain how this approach is used to trick the masses into believing that the performance of a parallel code is better than it really is.

[6 marks]